



국립 한국해양대학교
NATIONAL
KOREA MARITIME & OCEAN UNIVERSITY

친환경스마트조선기자재학과

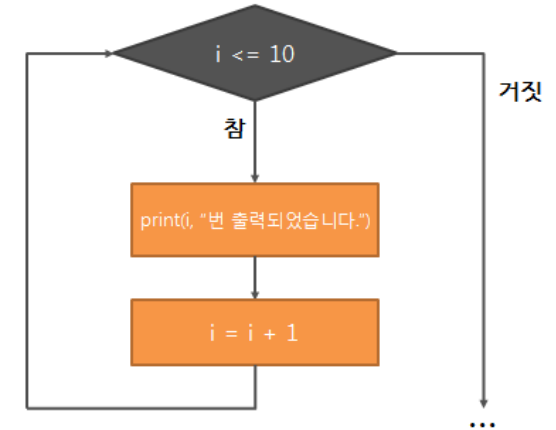
데이터처리특론



같은 일을 되풀이하는 방법

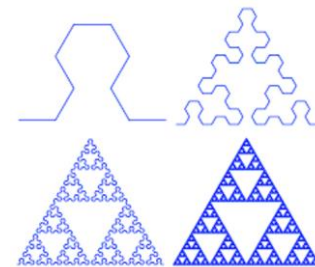
반복(iteration)

- for나 while 등의 반복문 이용
- 보통은 수행속도가 빠름
- 문제에 따라 프로그램 작성이 어려울 수 있음



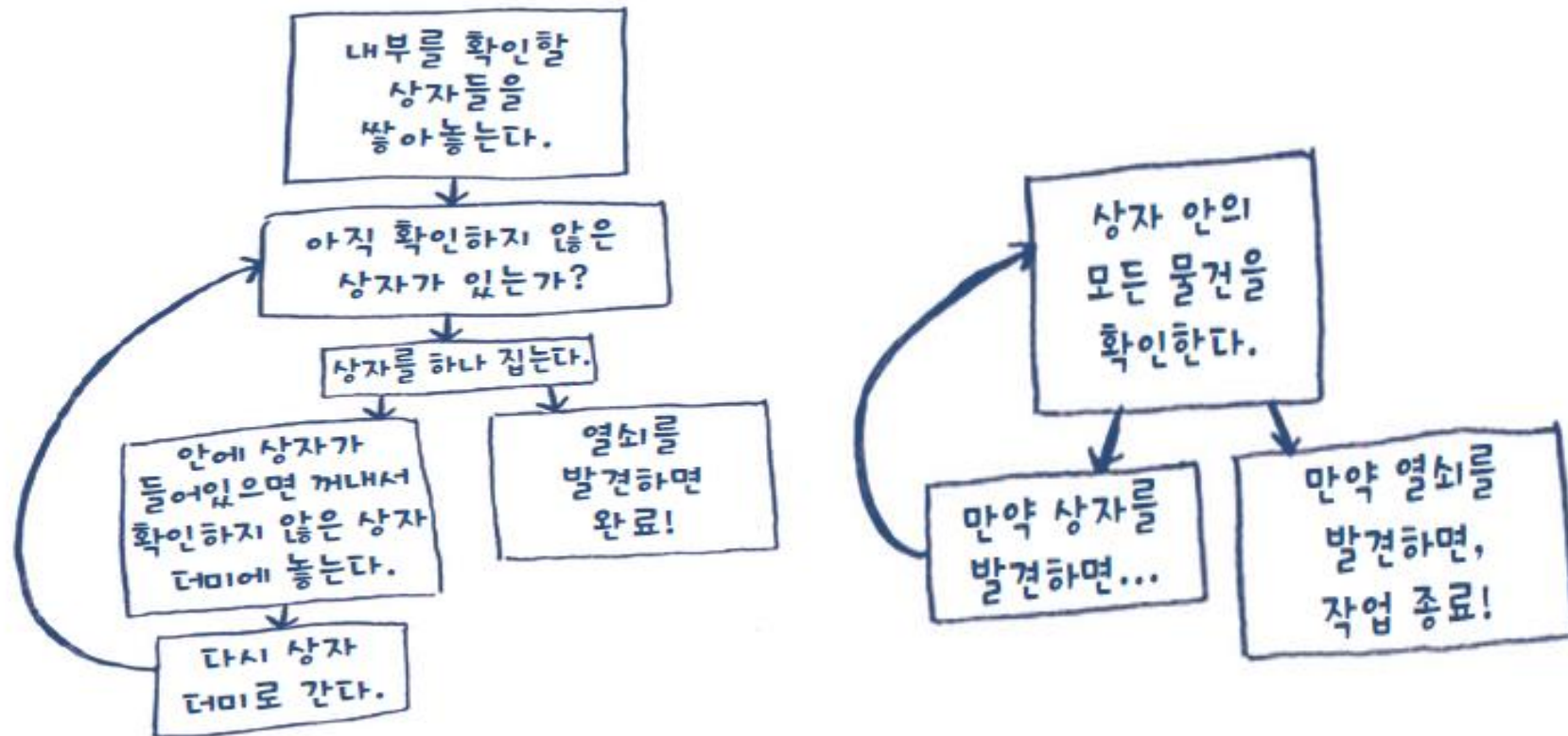
순환(recursion)

- 알고리즘이나 함수가 수행 도중에 자기 자신을 다시 호출하여 문제를 해결하는 기법
- 함수 호출의 오버헤드가 있음
- 간결한 코딩이 가능



재귀(Recursion)

- 풀이를 더 명확하게 만들
- 상황에 따라 반복문과 재귀 중 적절한 방법을 사용
- Ex) 상자들 속에서 어떤 물건(열쇠)을 찾는 경우



While & Recursion

While 사용

```
def look_for_key(main_box):  
    pile = main_box.make_a_pile_to_look_through()  
    while pile is not empty:  
        box = pile.grab_a_box()  
        for item in box:  
            if item.is_a_box():  
                pile.append(item)  
            elif item.is_a_key():  
                print "열쇠를 찾았어요!"
```

재귀 사용

```
def look_for_key(box):  
    for item in box:  
        if item.is_a_box():  
            look_for_key(item)  
        elif item.is_a_key():  
            print "열쇠를 찾았어요!"
```

반복!

Base & Recursive Case

기본 단계(Base Case)와 재귀 단계(Recursive Case)

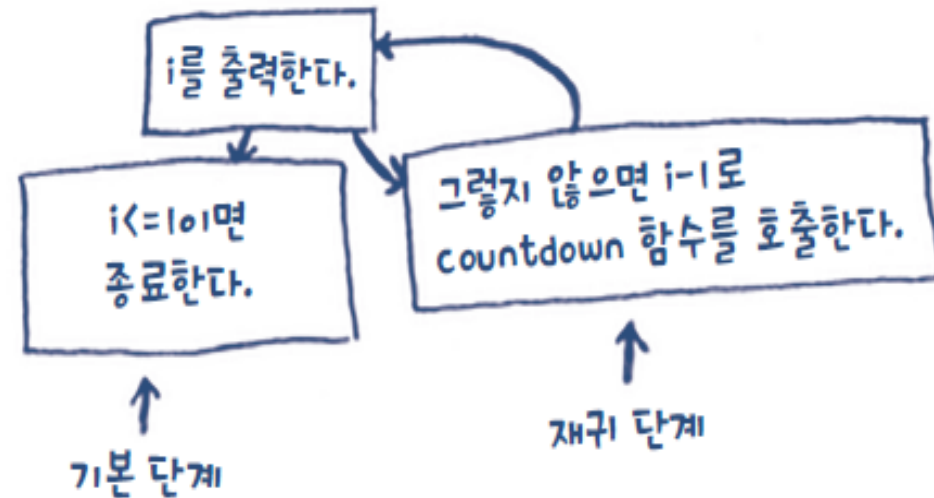
- 모든 재귀 함수는 이 두 부분으로 나뉨
- 재귀 함수는 실수로 무한반복 함수 만들기 쉬움
 - 재귀 단계: 함수가 자기 자신을 호출하는 부분
 - 기본 단계: 함수가 스스로를 다시 호출하지 않도록 하는 부분

```

def countdown(i):
    print i
    if i <= 1:
        return
    else:
        countdown(i-1)
    
```

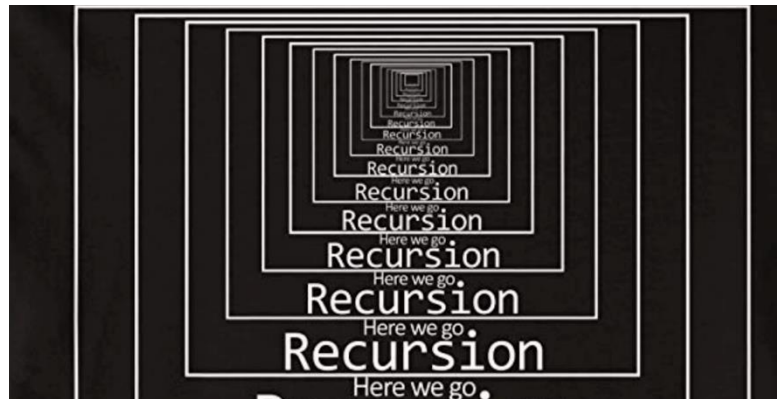
The code is annotated with green dashed boxes and arrows:

- A green dashed box labeled "기본 단계" (Base Case) points to the `if i <= 1:` condition.
- A green dashed box labeled "재귀 단계" (Recursive Case) points to the `countdown(i-1)` recursive call.



재귀(순환)

- 어떠한 것을 정의할 때 자기 자신을 참조하는 것
- 자기연급과도 관련된 재귀는 언어학에서 논리학에 이르기까지 다양한 분야에서 연구되는 주제
- 특히 컴퓨터 과학과 수학에서, 재귀는 함수가 자신의 정의에 의해 정의될 때의 개념
- 재귀 함수(Recursion function)
 - 정의 단계에서 자신을 재참조하는 함수
 - 어떤 사건이 자신을 포함하고 다시 자기 자신을 사용하여 정의될 때 재귀적(recursive)이라고 함.
 - 일단 무언가를 설명할 때 자기를 포함한 것이라고 이해하면 편함.



재귀 = 자기 호출

- 자신과 성격은 똑같지만 크기만 작은 r것을 호출
- 복잡한 문제도 간명하게 볼 수 있게 한다
 - 예) 탐색, 정렬, 수열, ...
- 잘 쓰면 보약
 - 정렬, 탐색, ...
- 잘못 쓰면 독약
 - 피보나치수 구하기, 최적 행렬곱 경로, ...

순환이 적합한 경우

정의자체가 순환적으로 되어 있는 문제나 자료구조

- 팩토리얼 구하기
$$n! = \begin{cases} 1 & n=1 \\ n*(n-1)! & n>1 \end{cases}$$
- 피보나치 수열
$$fib(n) = \begin{cases} 0 & \text{if } n=0 \\ 1 & \text{if } n=1 \\ fib(n-2) + fib(n-1) & \text{otherwise} \end{cases}$$
- 이항 계수, 하노이의 탑, 이진 탐색, 등

Factorial

반복 구조

$$n! = 1 \cdot 2 \cdot 3 \cdot \dots \cdot n$$

```
def factorial_iter(n) :  
    result = 1  
    for k in range(1, n+1) :  
        result = result * k  
    return result
```

순환 구조

$$n! = \begin{cases} 1 & n=1 \\ n \cdot (n-1)! & n>1 \end{cases}$$

```
def factorial(n) :  
    if n == 1 :  
        return 1  
    else :  
        return n * factorial(n-1)
```

Factorial

순환적인 함수 호출 순서

factorial(3) = 3 * factorial(2)
 = 3 * 2 * factorial(1)
 = 3 * 2 * 1
 = 3 * 2
 = 6

n=3

```

def factorial(n) :
    if n == 1 : return 1
    else : return n * factorial(n - 1)
    
```

⑤ 6반환

n=2

```

def factorial(n) :
    if n == 1 : return 1
    else : return n * factorial(n - 1)
    
```

④ 2반환

n=1

```

def factorial(n) :
    if n == 1 : return 1
    else : return n * factorial(n - 1)
    
```

③ 1반환

Factorial

Spyder (Python 3.11)

File Edit Search Source Run Debug Consoles Projects Tools **View** Help

C:\Users\wisadora\Documents\2024\2024-1\한국해양대학교\자료구조\강의자료\3주차\factorial_iter.py

untitled1.py x factorial_iter.py x

```

1 def factorial_iter(n):
2     result = 1
3     for k in range(1, n+1):
4         result = result * k
5     return result
6
7 print(factorial_iter(3))
    
```

Debug Consoles Projects Tools View Help

Debug	Ctrl+F5
Debug cell	Alt+Shift+Return
Step	Ctrl+F10
Step Into	Ctrl+F11
Step Return	Ctrl+Shift+F11
Continue	Ctrl+F12
Stop	Ctrl+Shift+F12
Set/Clear breakpoint	F12
Set/Edit conditional breakpoint	Shift+F12
Clear breakpoints in all files	
List breakpoints	

Spyder (Python 3.11)

File Edit Search Source Run Debug Consoles Projects Tools View Help

C:\Users\wisadora\Documents\2024\2024-1\한국해양대학교\자료구조\강의자료\3주차\factorial_recu.py

untitled1.py x factorial_iter.py x factorial_recu.py x

```

1 def factorial(n):
2     if n == 1:
3         return 1
4     else:
5         return n*factorial(n-1)
6
7 print(factorial(3))
    
```

conda (Python 3.11.4) Completions: conda LSP: Python Line 7, Col 20 UTF-8 CRLF RW Mem 39%

View – Panes – Variable Explorer

Factorial

fact(n) :

```
tmp ← 1  
for i ← 2 to n  
    tmp ← i * tmp  
return tmp
```

← 비재귀

fact(n) :

```
if (n == 1) return 1  
else return n * fact(n-1)
```

← 재귀

- 일반적으로 순환은 프로그램(알고리즘)의 가독성을 높일 수 있는 장점을 갖지만 시스템 스택을 사용하기 때문에 메모리 사용 측면에서 비효율적임
- 반복문으로 변환하기 어려운 순환도 존재하며, 억지로 반복문으로 변환하는 경우 프로그래머가 시스템 스택의 수행을 처리해야 하므로 프로그램이 매우 복잡해질 수 밖에 없음
- 반복문으로 변환된 프로그램의 수행 속도가 순환으로 구현된 프로그램보다 항상 빠르다는 보장 없음

재귀가 치명적인 예

- 피보나치 수열

$$F_0 = 0$$

$$F_1 = 1$$

$$F_n = F_{n-1} + F_{n-2} \quad (n \in \{2, 3, 4, \dots\})$$

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, 987, 1597, 2584, 4181, 6765, 10946, ...

fib(n):

if (n=1 or n=2)

return 1

else

return fib(n-1) + fib(n-2)

재귀

fib_fast(n):

f[1] ← f[2] ← 1 ◀ "f[2] ← 1"과 "f[1] ← 1"을 한꺼번에 적어놓은 것

for i ← 3 to n

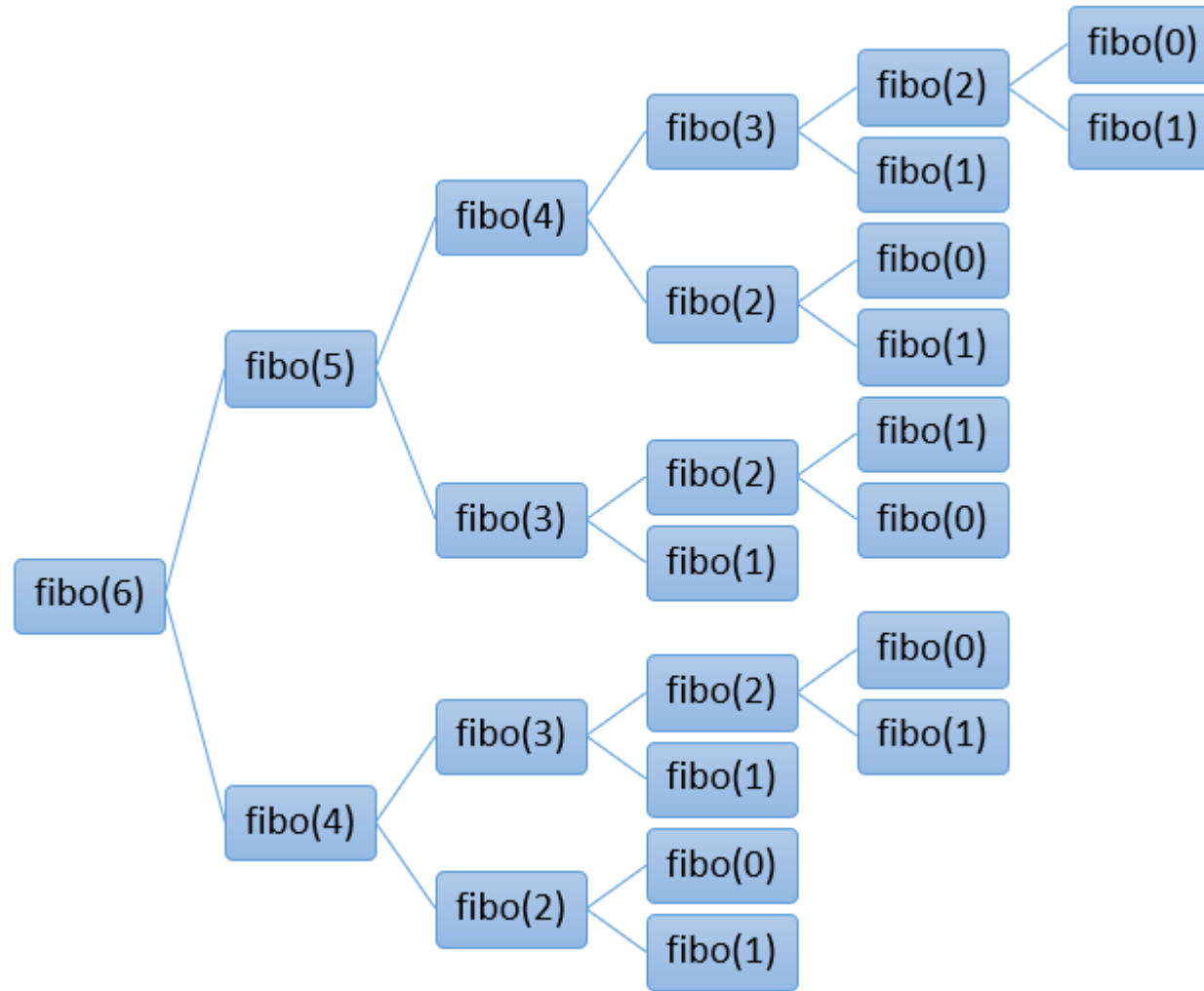
f[i] ← f[i-1] + f[i-2]

return f[n]

비재귀

Simple, but **terrible!**

Fibonacci



Fibonacci

```
import math
import time

def fib(n):
    if(n==1 or n==2):
        return 1
    else:
        return fib(n-1) + fib(n-2)

start = time.time()
print(fib(40))
end = time.time()
print(f"{end - start:.5f} sec")
```

```
import math
import time

def fib_rep(n):
    result = []
    a , b = 1, 1
    for i in range(n):
        result.append(a)
        a,b = b, a+b
    return result

start = time.time()
fibonacci = []
fibonacci = fib_rep(50)
print(fibonacci)
end = time.time()
print(f"{end - start:.5f} sec")
```

Fibonacci

fib(50) – 36초

PC: Pentium 3GHz

fib(66) – 하루 정도

fib(100) – 3만5천년 정도

fib(136) – 1조년 초과

지수함수적 중복 호출로 인해 이런 치명적인 비효율이 발생한다

```
fib(n) :  
    t[1] ← t[2] ← 1  
    for i ← 3 to n  
        t[i] ← t[i-1] + t[i-2]  
    return t[n]
```

천만 분의 1초도 안 걸린다

개선 방법

An exponential algorithm

A polynomial algorithm

행렬을 이용한 연산

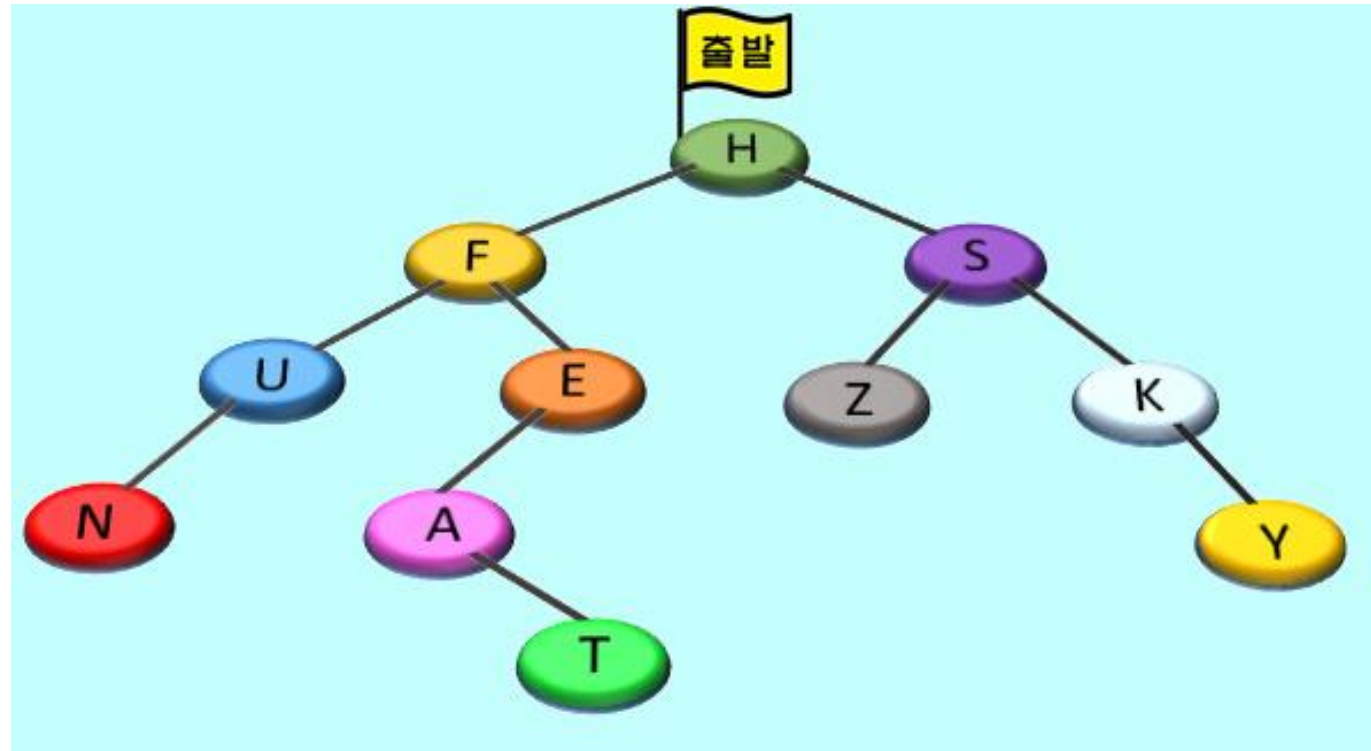
캐시를 사용하는 방법(lru_cache)

반복문을 사용하는 방법

비네의 식(Binet's formula)을 사용하는 방법

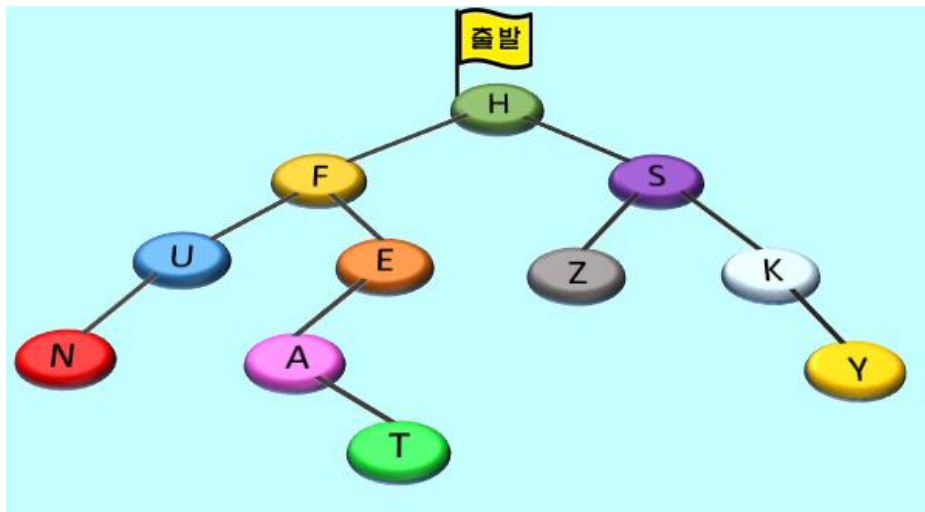
Memoization

[예제] 남태평양에 있는 어느 나라에 11개의 섬이 그림과 같이 다리로 연결되어 있다. 이 나라의 관광청에서는 관광객들이 모든 11개의 섬들의 방문 순서가 다른 3개의 관광코스를 개설



각 코스의 관광은 섬 H에서 시작하며, 관광청에서는 각 관광 코스의 방문 순서를 A, B, C 규칙 하에 만들었다.

A-코스: 섬에 도착하면 항상 도착한 섬을 먼저 관광하고, 그 다음엔 왼쪽 섬으로 관광을 진행하고 왼쪽 방향의 모든 섬들을 방문한 후에 오른쪽 섬으로 관광을 진행

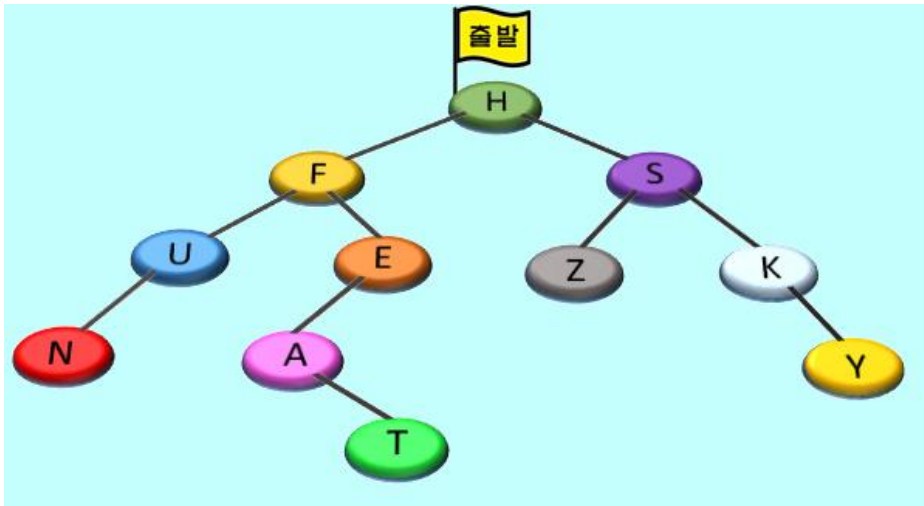


```

def A_course(n): # A-코스
    if n != None:
        print(n.name, '-> ', end='') # 섬 n 방문
        A_course(n.left) # n의 왼쪽으로 진행
        A_course(n.right) # n의 오른쪽으로 진행
    
```

B-코스: 섬에 도착하면 도착한 섬의 관광을 미루고, 먼저 왼쪽 섬으로 관광을 진행하고 왼쪽 방향의 모든 섬들을 방문한 후에 돌아와서 섬을 관광한다. 그 다음엔 오른쪽 섬으로 관광을 진행

섬 H는 왼쪽 방향의 섬 F, U, N, E, A, T를 모두 관광한 다음에 관광

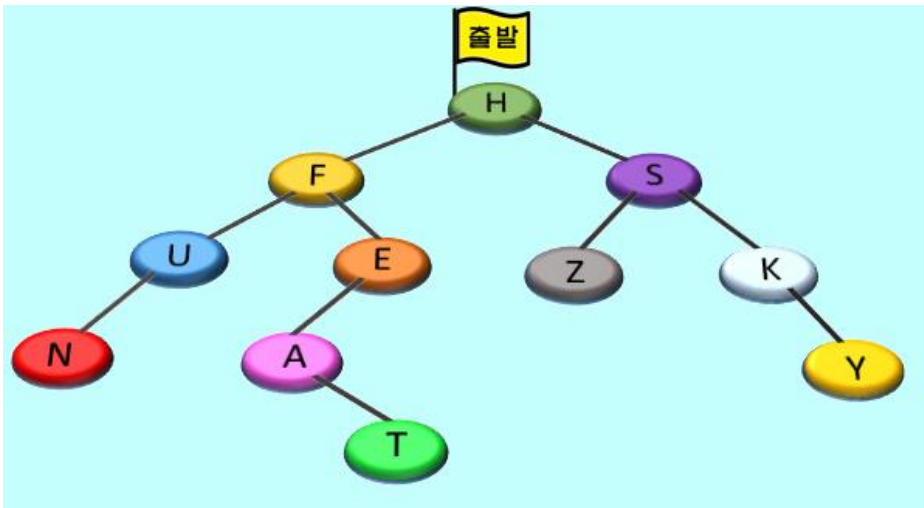


```

def B_course(n): # B-코스
    if n != None:
        B_course(n.left)      # n의 왼쪽으로 진행
        print(n.name, '-> ', end='') # 섬 n 방문
        B_course(n.right)     # n의 오른쪽으로 진행
    
```

C-코스: 섬에 도착하면 도착한 섬의 관광을 미루고, 먼저 왼쪽 섬으로 관광을 진행하고 왼쪽 방향의 모든 섬들을 관광한 후에 돌아와서, 오른쪽 섬으로 관광을 진행하고 오른쪽 방향의 모든 섬들을 관광한 후에 돌아와서, 드디어 섬을 관광

섬 H는 왼쪽 방향의 섬 F, U, N, E, A, T를 모두 관광하고 오른쪽 방향의 모든 섬 S, Z, K, Y를 관광한 다음에 마지막으로 관광



```

def C_course(n): # C-코스
    if n != None:
        C_course(n.left)      # n의 왼쪽으로 진행
        C_course(n.right)     # n의 오른쪽으로 진행
        print(n.name, '-> ', end='') # 섬 n 방문
    
```

```

01 class Node:
02     def __init__(self, name, left=None, right=None): # 섬 생성자
03         self.name = name
04         self.left = left
05         self.right = right
06
07 def map(): # 지도 만들기
08     n1 = Node('H')
09     n2 = Node('F')
10     n3 = Node('S')
11     n4 = Node('U')
12     n5 = Node('E')
13     n6 = Node('Z')
14     n7 = Node('K')
15     n8 = Node('N')
16     n9 = Node('A')
17     n10 = Node('Y')
18     n11 = Node('T')
19
20     n1.left = n2
21     n1.right = n3
22     n2.left = n4
23     n2.right = n5
24     n3.left = n6
25     n3.right = n7
26     n4.left = n8
27     n5.left = n9
28     n7.right = n10
29     n9.right = n11
30     return n1 # 시작 섬 리턴
    
```

11개의 섬 만들기

11개의 섬 교량으로 잇기

```

31 def A_course(n): # A-코스
32     if n != None:
33         print(n.name, '-> ', end='') # 섬 n 방문
34         A_course(n.left)           # n의 왼쪽으로 진행
35         A_course(n.right)          # n의 오른쪽으로 진행
36
37 def B_course(n): # B-코스
38     if n != None:
39         B_course(n.left)           # n의 왼쪽으로 진행
40         print(n.name, '-> ', end='') # 섬 n 방문
41         B_course(n.right)          # n의 오른쪽으로 진행
42
43 def C_course(n): # C-코스
44     if n != None:
45         C_course(n.left)           # n의 왼쪽으로 진행
46         C_course(n.right)          # n의 오른쪽으로 진행
47         print(n.name, '-> ', end='') # 섬 n 방문
    
```

```

49 start = map() # 시작 섬을 n1으로
50 print('A-코스:\t', end='')
51 A_course(start)
52 print('\nB-코스:\t', end='')
53 B_course(start)
54 print('\nC-코스:\t', end='')
55 C_course(start)
    
```

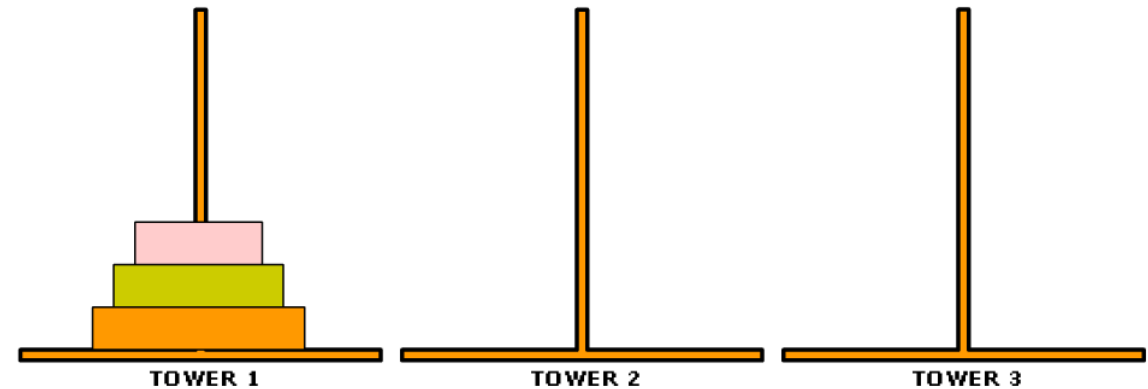
관광 결과

A-코스:	H	->	F	->	U	->	N	->	E	->	A	->	T	->	S	->	Z	->	K	->	Y	->
B-코스:	N	->	U	->	F	->	A	->	T	->	E	->	H	->	Z	->	S	->	K	->	Y	->
C-코스:	N	->	U	->	T	->	A	->	E	->	F	->	Z	->	Y	->	K	->	S	->	H	->

Tower of Hanoi

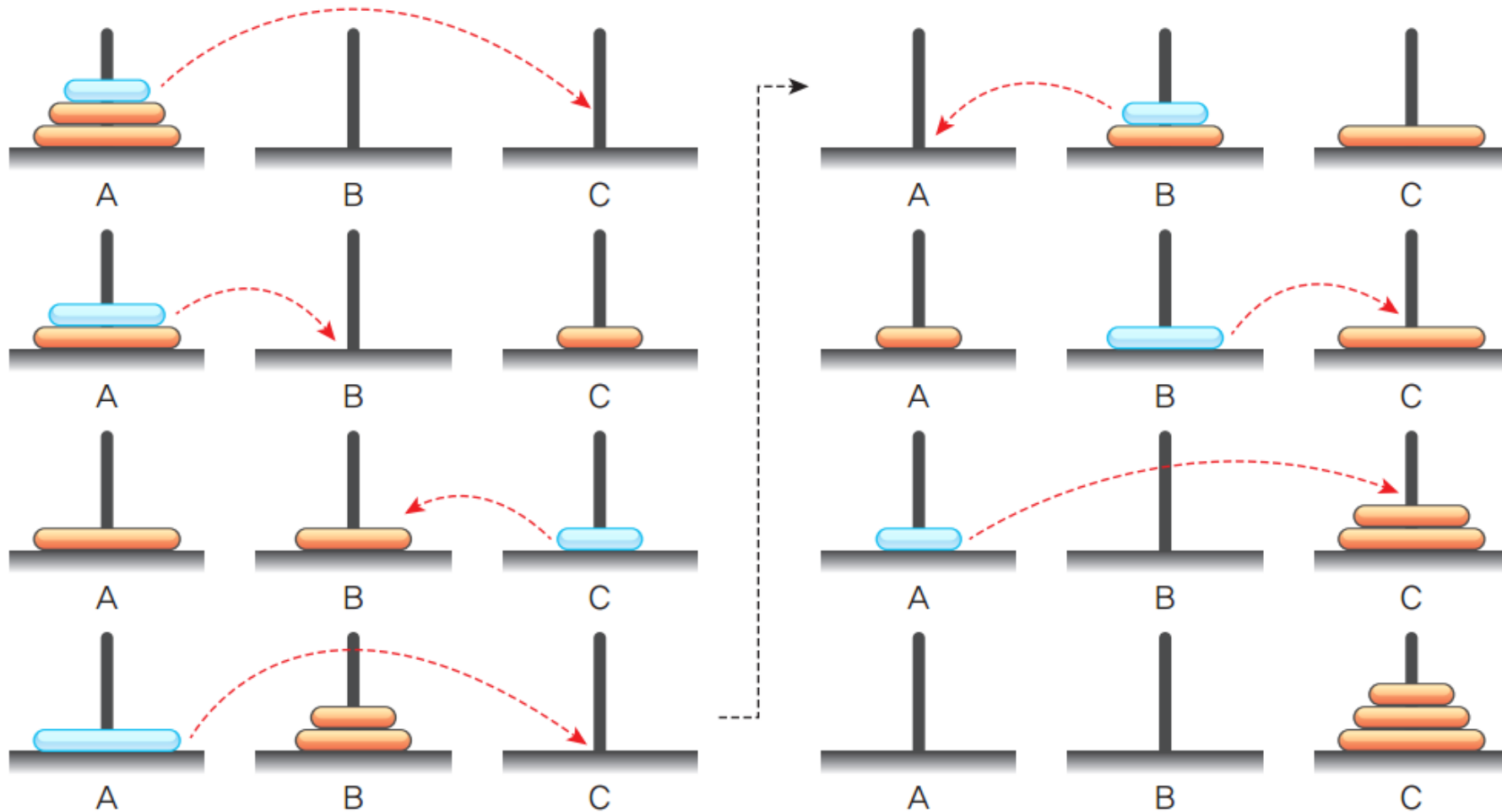
하노이 탑

- 세 개의 기둥과 이 기둥에 꽂을 수 있는 크기가 다양한 원판들이 있고, 시작하기 전에는 한 기둥에 원판들이 작은 것이 위에 있도록 순서대로 쌓여 있다.
- 다음 조건을 만족시키면서, 한 기둥에 꽂힌 원판들을 그 순서 그대로 다른 기둥으로 옮겨서 다시 쌓는 것이다
 1. 한 번에 한개의 원판만 옮길 수 있다.
 2. 가장 위에 있는 원판만 이동할 수 있다.
 3. 큰 원판이 작은 원판 위에 있어서는 안 된다.



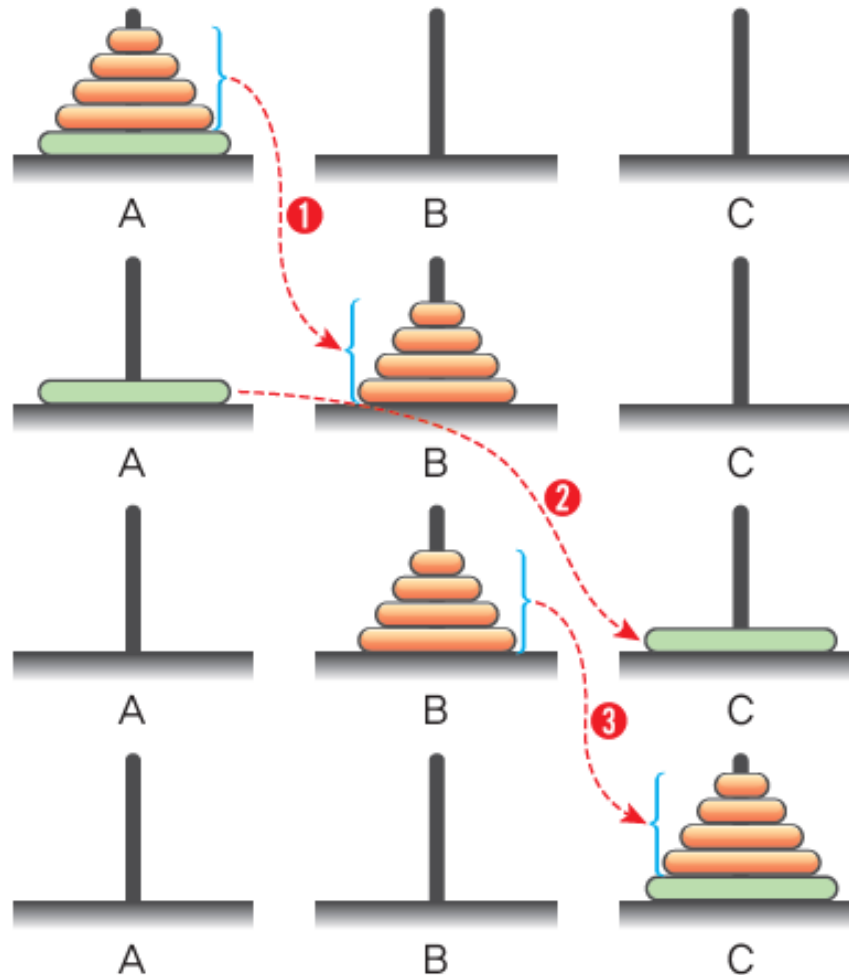
Tower of Hanoi

$n = 3$



Tower of Hanoi

일반적인 경우



① 단계: A에 있는 $n-1$ 개의 원판을 C를 임시 막대로 이용해서 B로 이동

② 단계: A에 남은 하나의 원판을 C로 이동

③ 단계: B에 있는 $n-1$ 개의 원판을 A를 임시 막대로 이용해서 C로 이동

Tower of Hanoi

구현

```
def hanoi_tower(n, fr, tmp, to) :           # Hanoi Tower 순환 함수

    if (n == 1) :                          # 종료 조건
        print("원판 1: %s --> %s" % (fr, to))  # 가장 작은 원판을 옮김
    else :
        hanoi_tower(n - 1, fr, to, tmp)      # n-1개를 to를 이용해 tmp로
        print("원판 %d: %s --> %s" % (n, fr, to))  # 하나의 원판을 옮김
        hanoi_tower(n - 1, tmp, fr, to)      # n-1개를 fr을 이용해 to로
```

```
hanoi_tower(4, 'A', 'B', 'C')             # 4개의 원판이 있는 경우
```

Tower of Hanoi

결과

The screenshot shows the Spyder Python IDE with a file named `hanoi.py` open. The code implements the Tower of Hanoi algorithm for 4 disks, moving them from source 'A' to target 'C' using an auxiliary disk 'B'. The console displays the sequence of moves, with each move labeled by the disk number and the source and target disks. The output is as follows:

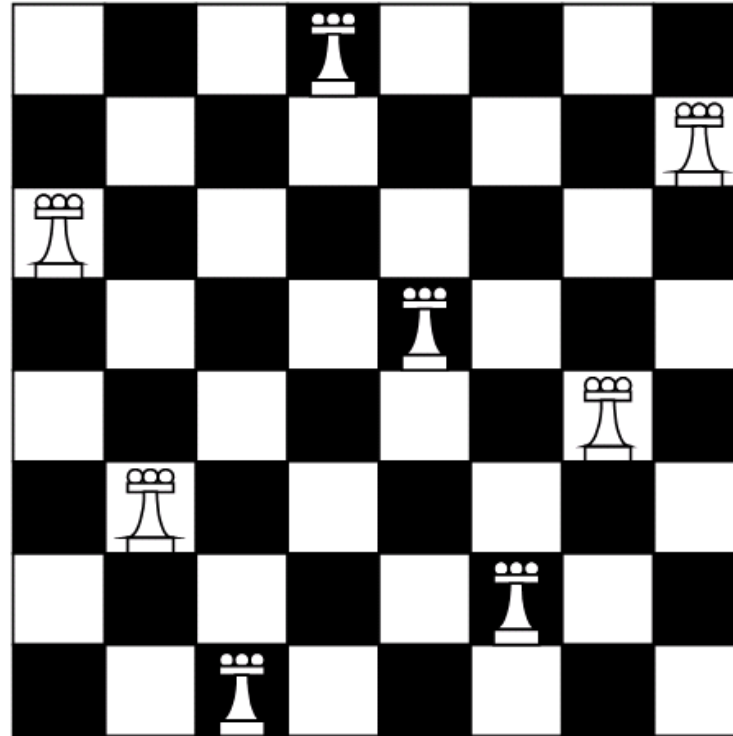
```

원판 1: A -> B
원판 2: A -> C
원판 1: B -> C
원판 3: A -> B
원판 1: C -> A
원판 2: C -> B
원판 1: A -> B
원판 4: A -> C
원판 1: B -> C
원판 2: B -> A
원판 1: C -> A
원판 3: B -> C
원판 1: A -> B
원판 2: A -> C
원판 1: B -> C
  
```

Below the console output, the text `In [31]:` is visible, followed by the Korean text `4개의 원판 영상` (4-disk image).

N-Queen Problem

N-Queen 문제

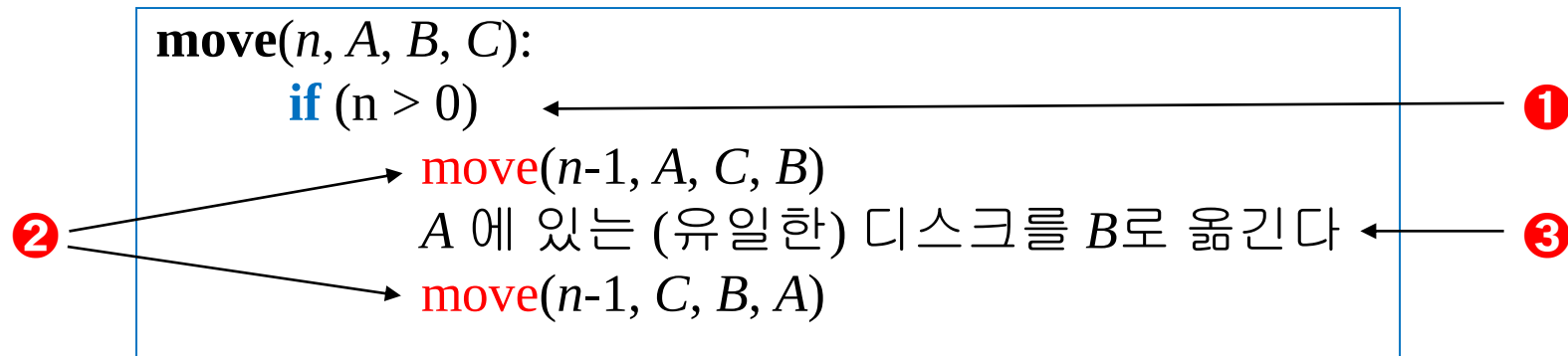


[백준 9663번] N-Queen in python

재귀 알고리즘의 필수 구비 조건

- ❶ 경계 조건 Base Condition(또는 종료 조건): 재귀 호출이 반복되다 궁극적으로 끝나는 조건
- ❷ 재귀 호출
- ❸ 관계: 닮음꼴 작은 문제(들)와 본 문제 간의 관계를 나타내는 부분

하노이 탑의 예



경계(종료) 조건이 없으면?

Base case가 없으면?

fib(n) :

return (**fib**($n-1$)+**fib**($n-2$))

move(n, A, B, C) :

Base case가 없으면?

move($n-1, A, C, B$)

A 에 있는 (유일한) 디스크를 B 로 옮긴다

move($n-1, C, B, A$)

Deduction & Induction



Mathematical induction

- 자연수에 관한 명제 $P(n)$ 이 모든 자연수(또는, 어떤 자연수보다 큰 모든 자연수)에 대하여 성립함을 보이는 증명법이다. 증명은 두 부분으로 구성되는데, 첫 번째 부분은 최소원 $n = n_0$ 에 대해 $P(n_0)$ 가 성립함을 보이는 부분이며, 두 번째 부분에서는 어떤 자연수 k 에 대해 $P(k)$ 가 성립한다는 가정 하에 $P(k+1)$ 또한 성립함을 보이게 된다.
- 첫 번째 부분을 basis step, 두 번째 n 에 대한 임의의 상수 k 를 가정하는 부분을 assumption step, 세 번째 $k+1$ 또한 n 에서 성립함을 보이는 부분을 inductive step, 네 번째 결론을 도출하는 부분을 conclusion step이라 한다.

1) $P(1)$ 이 성립
2) $P(n)$ 이 성립하면 $P(n+1)$ 이 성립

1) $P(1)$ 이 성립한다.
2) $P(1), P(2), \dots, P(n)$ 이 모두 성립하면 $P(n+1)$ 이 성립한다.

하노이 타워 문제와 수학적 귀납법

$T(n)$: 하노이 탑 알고리즘이 n 개의 원반을 옮기는 데 필요한 이동(원반 하나를 옮기는 것)의 총 횟수

$$T(n) = 2^n - 1$$

<증명: 수학적 귀납법>

- i) (경계조건 만족) 우선 0개를 옮기는 데는 이동 횟수가 0번이다. $T(0) = 2^0 - 1 = 0$ 이므로 만족한다.
- ii) (귀납적 가정) k 개의 원반을 옮기는 데 필요한 이동 횟수 $T(k) = 2^k - 1$ 이라 가정하자.
- iii) (귀납적 전개: $k+1$ 개의 원반을 옮기는 데 필요한 이동 횟수 $T(k+1) = 2^{k+1} - 1$ 이 됨을 보인다)
 $k+1$ 개의 원반을 옮기는 작업은 원반 k 개를 옮기는 작업 두 번, 원반 1개를 옮기는 작업 한 번으로 구성되므로 다음이 성립한다.

$$T(k+1) = 2T(k) + 1 = 2(2^k - 1) + 1 = 2^{k+1} - 1$$

하노이 타워 문제와 수학적 귀납법

Fact: $T(n) = 2^n - 1$

$T(n)$: 하노이 탑 알고리즘이 n 개의 원반을 옮기는 데 필요한 이동(원반 하나를 옮기는 것)의 총 횟수

<증명>

경계 조건: $T(0) = 0 = 2^0 - 1$

귀납적 가정: $T(k) = 2^k - 1$ 라 가정하자

귀납적 전개:

$$\begin{aligned} T(k+1) &= 2 \cdot T(k) + 1 \\ &= 2(2^k - 1) + 1 \\ &= 2^{k+1} - 1 \end{aligned}$$

Reference



- IT CookBook, 쉽게 배우는 자료구조 with 파이썬, 문병로, 한빛 아카데미
- 파이썬으로 쉽게 배우는 자료구조, 최영규, 천인국, 생능출판사
- Do it! 자료구조와 함께 배우는 알고리즘 입문 - 파이썬 편, 시바타 보요, 이지스퍼블리싱
- 파이썬과 함께하는 자료구조의 이해, 양성봉, 생능출판사
- <https://wikidocs.net/book/9059>
- <https://leeza.tistory.com/34577>